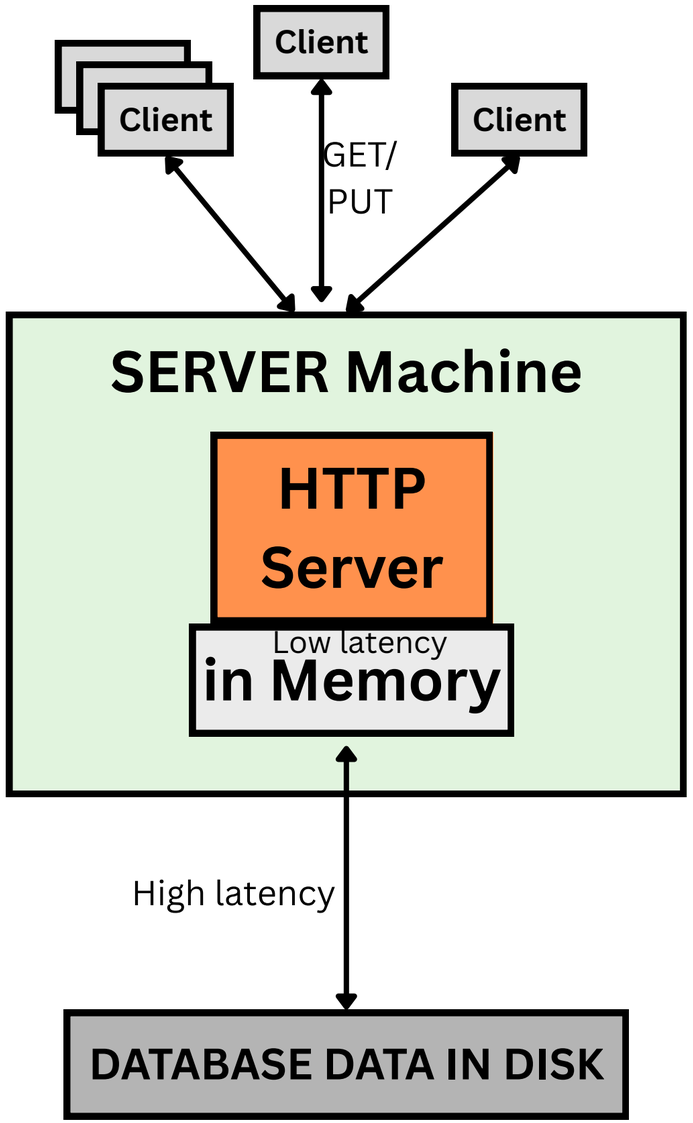
CS-744 Project Presentation
DESIGN AND ENGINEERING OF COMPUTING SYSTEMS

Name - Ashutosh Birla
Roll number - 24m0752
Professor - Mythili Vutukuru

1. System Components & Workflow

- **Core Components:** The system consists of a multi-threaded C++ Server, an in memory FIFO Cache for rapid data retrieval, and a persistent MySQL Database.
- **Load Generator:** A separate client emulates concurrent users as threads, generating closed loop traffic to stress test the system limits.
- **Read Requests (GET):** These requests first check the FIFO Cache. If found (Hit), data returns instantly (CPU Bound). If missing, data is fetched from MySQL and cached.
- **Write Requests (PUT/POST):** These requests bypass the read cache and write directly to the MySQL database, ensuring data persistence but incurring I/O latency.
- **Bottleneck Identification:** High cache hits saturate the CPU (fast path), while high cache misses or writes saturate the Disk I/O (slow path).



2. **Development process**

- Core Implementation: Wrote approximately 400 lines of C++ code, implementing the multi-threaded Server, the thread safe FIFO Cache logic, and the Load Generator. For SQL connector and library related issues taken few pieces of code from web.

- Third Party Integrations: Utilized cpp httplib for networking and mysql connector c++ for database operations to ensure robustness.

- Build & Config: Configured the CMake build system and MySQL connection setup by adapting standard documentation and online technical references.

3. **Load Generator Architecture:**

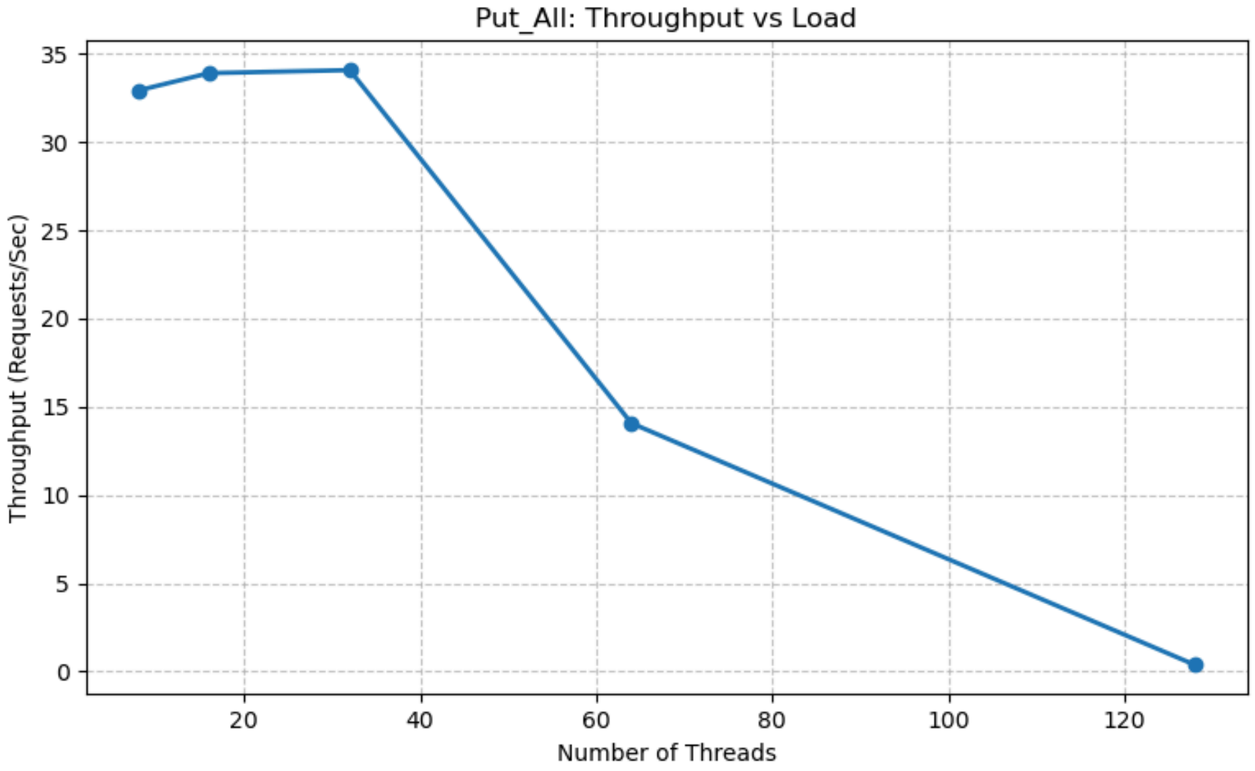
- Synchronous Multi Threading: Implemented a multi threaded client using cpp-httplib where each thread operates independently to simulate concurrent users.
- Closed Loop Model: Implemented a closed loop system, new requests are made only upon the completion of the previous request.

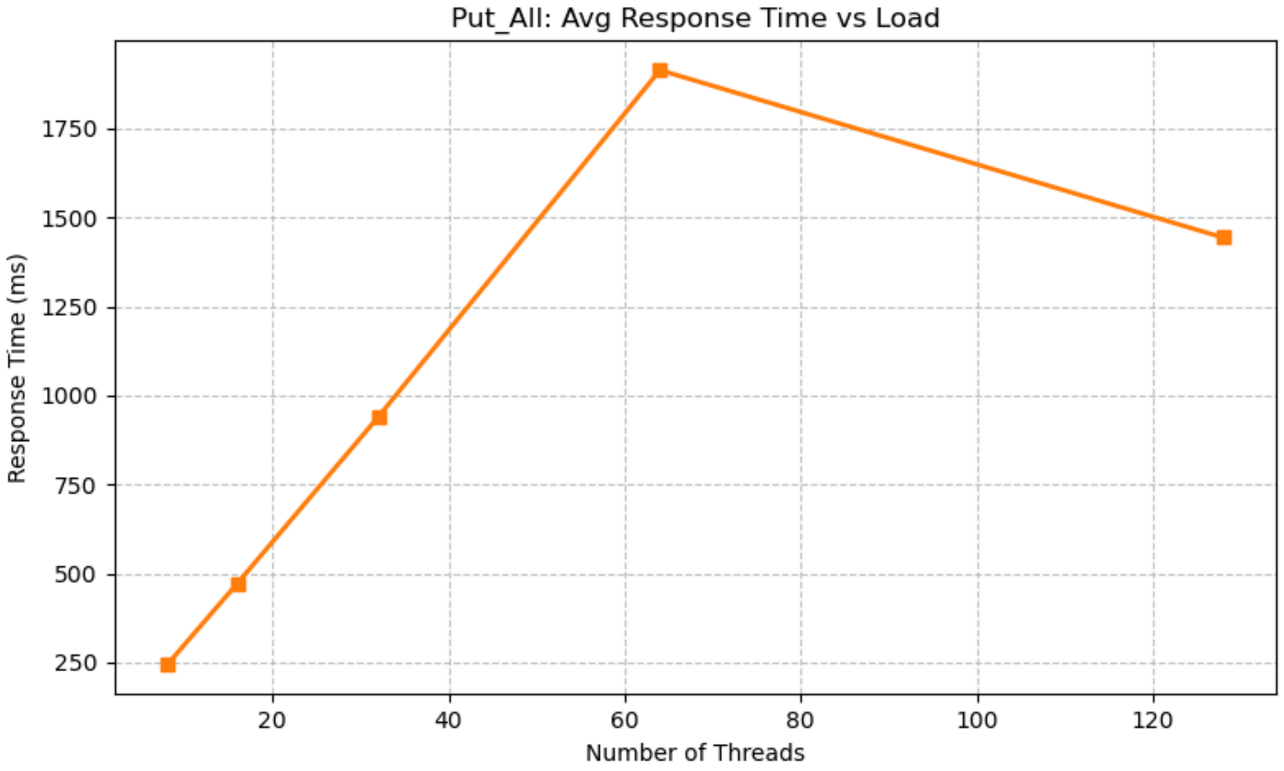
Generating High Stress:

- Multi Threading: I used multiple threads (e.g., 8, 16, 32) so many users can hit the server at the exact same time.
- Dedicated CPU Cores: I used taskset to give the Load Generator its own dedicated CPU cores. This ensures it saturate the server without competing for resources.
- Optimized Code: I removed slow operations like printing to the console (cout) during the test and used fast, thread safe random number generators to ensure the tool itself didn't become the bottleneck.

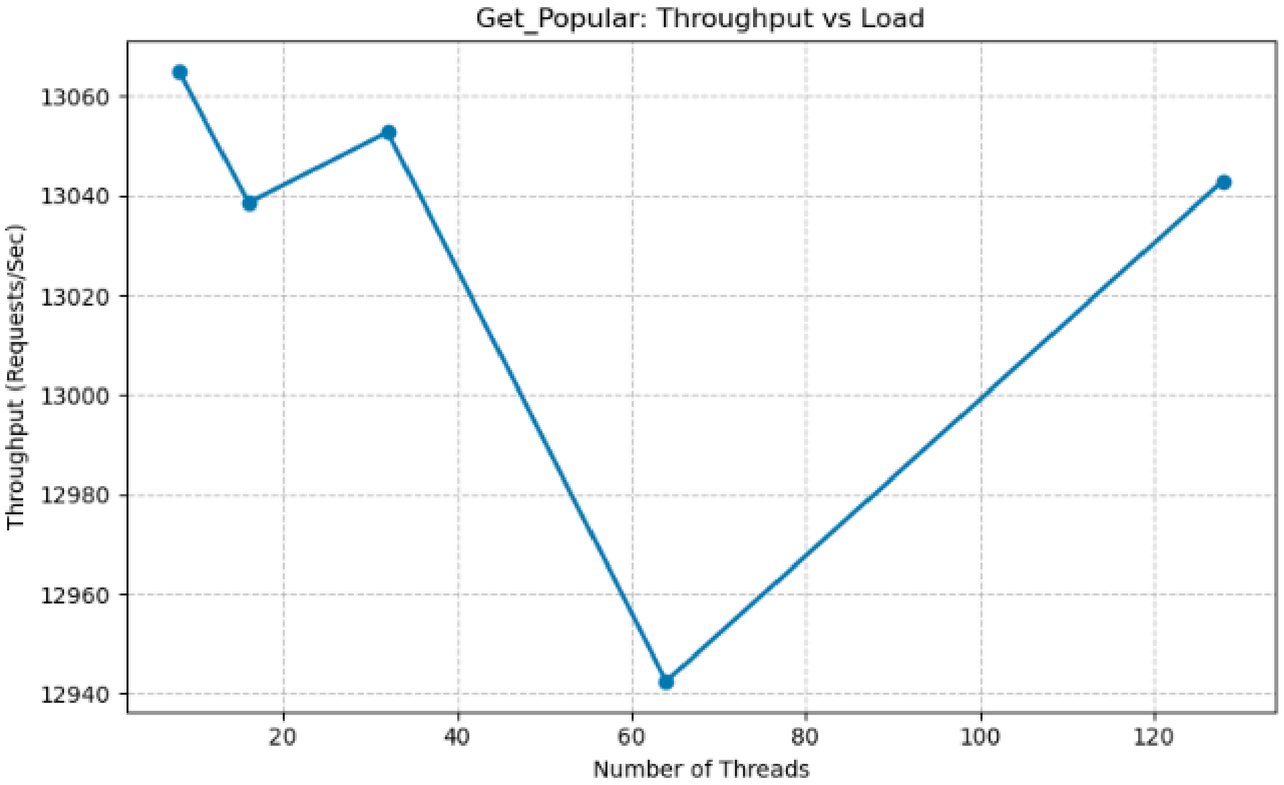
4. **Load test setup**

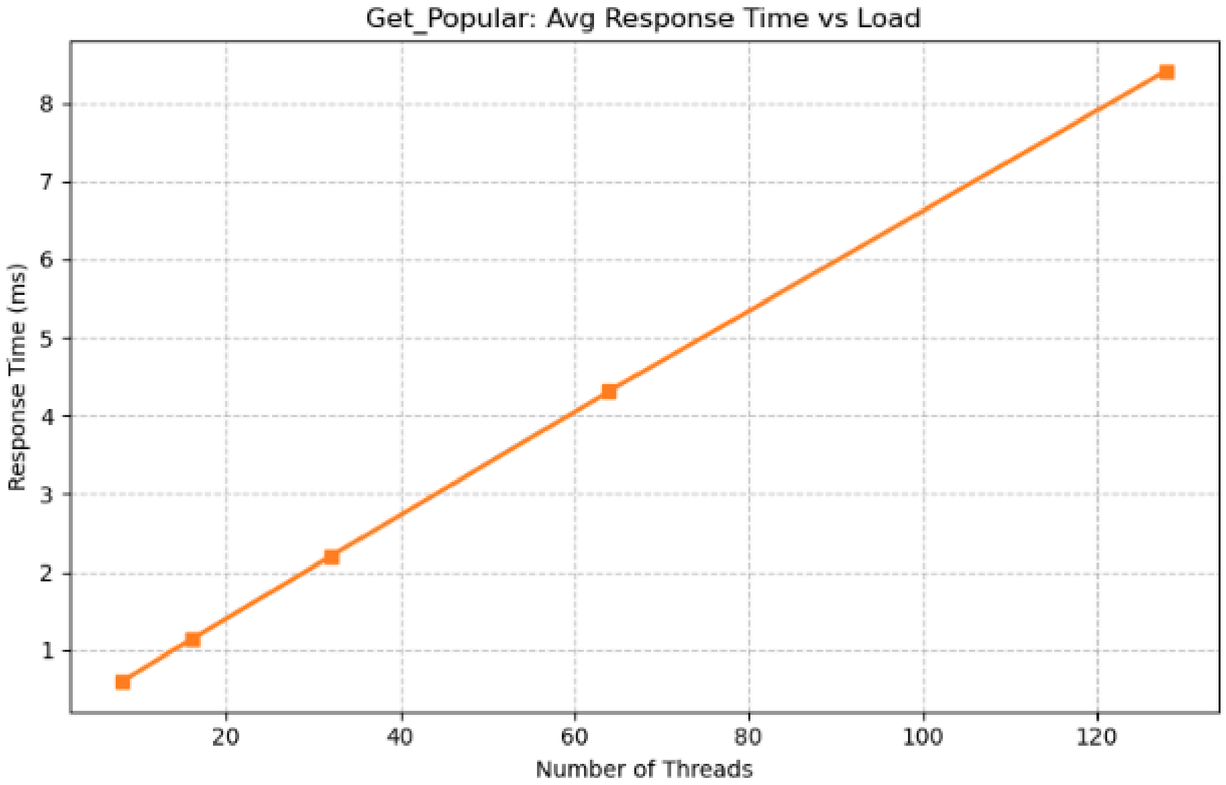
- **Resource Isolation**: Utilized a single multi core machine with strict CPU affinity (taskset) to prevent resource contention between components.
- **Core Pinning**: Assigned MySQL to Core 0, Server to Core 1 , and Load Generator to Cores 2-3 (Traffic Generation).
- **Workload Profiles**: Evaluated two distinct scenarios: "Put All" (Write Intensive) and "Get Popular" (Read Intensive/Cached).
- **Concurrency Scaling**: Incremented client threads (8, 16, 32, 64, 128) to identify the exact point of system saturation.
- **Experiment Duration**: Executed each test run for a fixed duration of 300 seconds.
- **Measured Metrics**: Average Throughput (Requests/Sec) and Average Response Time (Latency) at the client side.
- **Hardware Monitoring**: Correlated performance drops with hardware usage using mpstat (CPU utilization) and iostat (Disk utilization).





5. Load test graphs of throughput, latency for workload “Put all”





6. Load test graphs of throughput, latency for workload “Get Popular”